

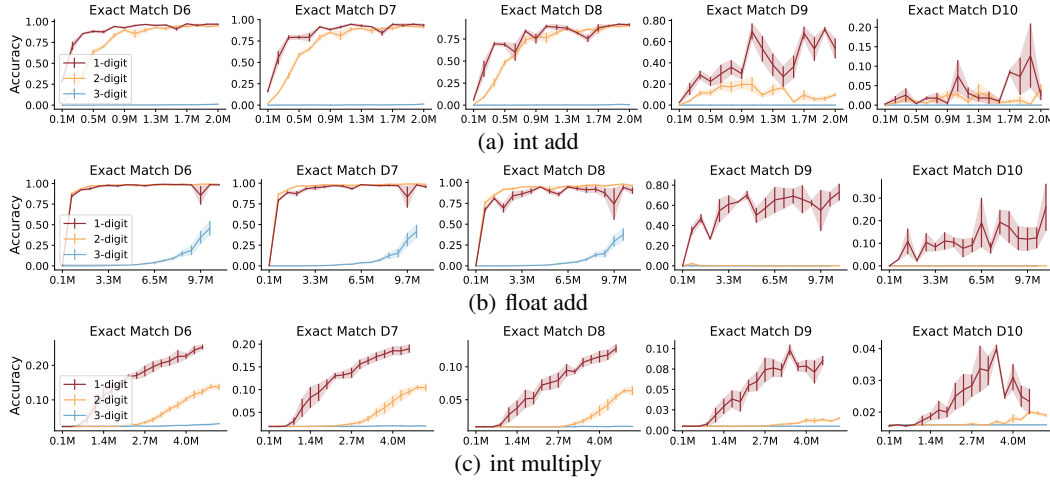


Figure 4: Accuracy of 0.9B models trained with 1-3 digit tokenizer on three tasks of integer addition, float addition and integer multiplication. Shadow shows the standard error. D_n means n digits. X-axis is the number of seen training samples.

of numbers in modern tokenizers has become more aligned. These tokenizers greedily segment a number from left to right into k -digit tokens until a remainder shorter than k digits is left, which is then segmented into a single token. Llama-2 uses a one-digit tokenizer, but all of the latest LLMs use a tokenizer with $k = 3$, which comes with an extended vocabulary for numbers. Additionally, Singh & Strouse (2024) discovers that just alternating the greedy direction from “left-to-right” to “right-to-left” (for integers) can improve performance of Llama-3 and GPT-4.

There is a growing tendency to expand the vocabulary size as the number of parameters in LLMs rapidly increases. Recent work has shown that a larger vocabulary is more suitable for larger LLMs (Tao et al., 2025) because longer tokens can encapsulate more complex and precise meanings for text tokens. However, numbers behave differently:

- The long-tail phenomenon (Raunak et al., 2020), common in text tokens, is not as pronounced for the number tokens. The distribution of number tokens is closer to a *uniform* distribution.
- Two smaller number tokens can always be combined into a valid new one (e.g., 3 and 7 form 37), which is not true for text tokens (e.g., “hello” and “hi” cannot form “hellohi”). So the number of possible number tokens grows exponentially as k increases, much faster than text tokens.
- The next token prediction of number tokens is harder than predicting the next text token because number prediction often involves calculation and operations, whereas word mapping tends to be more intuitive.

We trained 0.9B models on 1- to 8-digit length samples including integer addition, float addition, and integer multiplication, using **aligned** k -digit tokenizers where $k = 1, 2, 3$ (d in Figure 3). Figure 4 shows the in-domain performance of these models in the first three columns and their out-of-domain (OOD) performance in the last two columns, evaluated using the exact match metric.

From the figure, the one-digit tokenizer shows the best in-domain performance in these three tasks, while three-digit tokenizer exhibits poor performance. In out-of-domain tests, the one-digit tokenizer also exceeds the others by large margins. Tokenizers with an increasing number of digits significantly hinder sub-billion models’ NUPA. We also performed experiments on models of 3 different sizes including 0.1B, 0.9B, and 3B in Appendix A.4.2 and got similar results. Even as the model size increases, the performance of 2- or 3- digit tokenizer improves but remains either similar or worse than that of the one-digit tokenizers. For these experiments, we also report the digit match and length results in Appendix A.4.2 Figure 10 and 11, where one-digit tokenizer performs better both on digit learning (larger digit match) and length learning (less length). On the contrary, larger vocabularies significantly increase the model size requirements. In conclusion, we find **no evidence** to support the idea that increasing the vocabulary size improves NUPA performance.

Recently, Sathe et al. (2024) found that the “**random tokenizer**” (Kudo, 2018; Provilkov et al., 2020) which splits words like “Hello world” into variable tokens such as “He/llo/ world” or “Hell/o/ world” enhances reasoning by introducing variability in generation path. We also test it in number domain and find the random tokenizers consistently outperform their standard counterparts in length generalization, but **still fall short** of the performance achieved by the one-digit tokenizer. See the details in Appendix A.4.2.

3.2 SPECIALIZED PES ACT AS LENGTH REGULARIZERS

Previous work has suggested that PE could be the key factor (Zhou et al., 2024b) of length generalization. To further investigate whether the influence is specific on a certain task, we train 100M models with different PEs: RoPE (Su et al., 2024), NoPE (Kazemnejad et al., 2023) and Alibi (Press et al., 2022) on four tasks: *integer addition*, *float addition*, *fraction multiplication (easy)* and *scientific notation addition* respectively. Models are trained on 1-8 lengths (S and M range), then test them on full range (S to XL, 1-20). RoPE, widely used in Llama and its derivatives, is the most classic relative PE. Then Alibi, another relative PE, is proposed to address RoPE’s length overfitting issues. NoPE (transformers without PE, relying solely on the causal mask to encode the position information) offers a surprisingly easy way to achieve length generalization. Therefore, we compare these three typical PEs to evaluate the performance on NUPA.

Our results, presented in Figure 13 in Appendix A.4.3, align with conclusions from previous works. Alibi and NoPE demonstrate superior length generalization across various representations and tasks, indicating that the influence of PEs is relatively consistent across these common representations, tasks within the number domain.

Moreover, we aim to characterize further the mechanism underlying these differences. Specifically, we found that RoPE leads the model to learn a length-related *shortcut*, while Alibi and NoPE act as a form of *regularization* by avoiding this, thereby preventing length overfitting. For more details, please refer to the appendix A.4.3.

3.3 DATA FORMATS HELP DIGIT ALIGNMENT

A series of works have proposed specific data formats including reverse formatting, zero padding and index hints. **Reverse formatting** (Lee et al., 2024; Shen et al., 2023) presents numbers in reverse order from the least significant digit to the most significant one to align with the models’ autoregressive mechanism, simplifying the learning process for addition. **Zero padding** (Lee et al., 2024; Shen et al., 2023; Zhou et al., 2024b; Cho et al., 2024) adds leading zeros to numbers to standardize the lengths of operands, helping models align operands. **Index Hints** (Zhou et al., 2024a) explicitly incorporate positional information by prefixing each digit with its corresponding position index in both input and output sequences.

While previous work mainly focuses on integer addition or multiplication, we extend the techniques to various tasks in the NUPA Test of different number domains. To compare the effects of reverse formatting and zero padding, we demonstrate in Table 16 how the combination of reverse formatting and zero padding impacts length generalization. **Reverse formatting, zero padding, and their combination all outperform** vanilla formats in integer and float addition, while their performance is comparable to each other, suggesting that their functionality largely **overlaps**. Zero padding helps ensure proper alignment, while reverse formatting also plays a crucial role in maintaining alignment. The previously believed “*helping calculation*” function of reverse formatting is *minor*. As for index hint, we find it doesn’t work for our models. We discuss the details of these experiment results and the reasons in Appendix A.4.4.

3.4 DOES FINETUNING IMPROVE NUPA PERFORMANCE OF LLMs?

The existing techniques aimed at enhancing NUPA have rarely been applied to practical LLMs, mostly being tested on toy models and isolated tasks. This raises the question of whether it is possible to enhance the NUPA capabilities of large models through post-training finetuning. To explore this, we generate training sets (10^5 samples for each digit and each task) and validation sets for our NUPA tasks, ensuring no overlap with the original test set. We then use them to finetune a pretrained model. Specifically, we finetune a Meta-Llama-3.1-8B model with LoRA (Hu et al., 2022) (rank 128, $\alpha=32$) on a **mixed** training set comprising all of our NUPA tasks. Remarkably, we find only 800 steps training (about 50M training samples, $\ll$ 1 epoch) leads to significant improvement, as shown in Figure 2 with the finetuned model labeled as “Llama-8B-ft”. Though Llama-3.1-8B is not a strong baseline, this **finetuned version achieves much better performance**. For example, in max, max-hard, add-float and truediv tasks, this model even surpassed or matched GPT-4o, confirming our hypothesis: for many NUPA tasks, the model’s base capacity may not be the main limiting factor, but rather *the lack of numerical diversity and task variety* in the training data.

However, we also found that such finetuning does not provide much improvement on certain tasks, such as understanding digits. Furthermore, when we tried to incorporate the various tricks, such as modifying the model’s original PEs, tokenizers, or number formats, into an *already trained* model, these methods proved **ineffective**. When we altered the PE or adjusted the tokenization and representation of the model, the changes significantly disrupted the model’s original behavior, causing a substantial performance drop. This suggests that enhancing a model’s NUPA capabilities through post-training may require more revolutionary innovations beyond the current tricks. The detailed results of these attempts are presented in Table 18 in Appendix A.4.5.

4 IS CoT SUITABLE AND VALID FOR NUPA?

CoT has been proven to be effective in enhancing the capacity of LLMs both theoretically (Feng et al., 2023; Yang et al., 2024b) and experimentally (Wei et al., 2022; OpenAI, 2024b). Thus, we are also interested in whether CoT is the ultimate solution for improving NUPA. Due to the task and representation diversity in our benchmark, it is hard to cover all issues with a single form of CoT. So we adapt a special CoT form called Rule-Following CoT (Hu et al., 2024) (RF-CoT), where LLMs are trained to follow a provided code or pseudo-code that outlines the procedure to solve the task. RF-CoT is capable of handling any problem with a solving procedure that can be broken down into recurrences and basic unit operations, making it well-suited for our benchmark tasks. The detailed introduction with an example of RF-CoT can be found in Appendix A.5.1.

Table 3: Performance of RF CoT. “-” means exceeding context window limitation (2k tokens).

Exact Match # Digit	Add Float			Multiply Fraction			Max Scientific			Mod Integer		
	5	6	7	2	3	4	38	39	40	6	7	8
RF CoT	1.00 $\pm$.00	1.00 $\pm$.00	-	0.93 $\pm$.01	0.88 $\pm$.03	-	1.00 $\pm$.00	1.00 $\pm$.00	1.00 $\pm$.00	0.67 $\pm$.05	0.43 $\pm$.07	-
GPT-4o	0.78	0.66	0.49	0.53	0.20	0.00	0.37	0.46	0.36	0.01	0.00	0.00
Qwen2-72B	0.62	0.50	0.70	0.05	0.00	0.00	0.96	0.98	0.95	0.03	0.00	0.00
Llama-8B-ft	0.88 $\pm$.02	0.79 $\pm$.04	0.74 $\pm$.04	0.50 $\pm$.02	0.20 $\pm$.03	0.01 $\pm$.00	0.98 $\pm$.01	0.97 $\pm$.01	0.98 $\pm$.01	0.08 $\pm$.02	0.05 $\pm$.04	0.05 $\pm$.04

To evaluate the performance of this CoT method, we finetuned the LLaMA 3.1-8B model on a subset of the NUPA tasks with RF-CoT. During both training and testing, we set a context window of 2000 tokens, with any data exceeding this limit being ignored. Table 3 shows the performance on selected tasks. Accuracy and standard error for RF-CoT and finetuned Llama-3.1-8B are averaged over three runs. For GPT-4o and Qwen2, which are not finetuned, we report single-run accuracy without standard error. Within the context length limit, the rule-following finetuned LLaMA 3.1-8B significantly **outperformed** GPT-4o and Qwen2-72B as well as the one finetuned without RF-CoT in most situations.

Table 2: Average inference time.

	batchsize	sec / sample
RF CoT	128	5.625
Direct	128	0.371
Direct	256	0.336

However, it requires a significantly *longer context window* and causes much *slower inference speed* compared to directly generating the answer. As shown in Table 3, with the 2000-token limit, CoT can only handle fraction addition involving numbers up to three digits. We provide the maximal digit length within the 2k context window limitation for each task in Appendix A.5.2 to show the context window limitation for complex tasks. As for inference time, Table 2 demonstrates the average inference time for generating each sample using “RF-CoT” and “direct answer” during the NUPA Test where both experiments are operated on an A800 GPU. In the table, the “direct answer” with batch size 256 uses a similar amount of CUDA memory as RF-CoT with batch size 128. The RF-CoT method is approximately 17 times slower than directly generating the answer, causing an unsustainable burden for such a basic operation that is frequently encountered in solving real-world problems, especially considering that number calculations may only account for a small part of a complex, real-world reasoning problem (such as analyzing a financial report).

5 RELATED WORK

We have discussed some related work in the corresponding section. This section highlights some other studies related to NUPA in language models.

Numerical understanding in natural language comprehension Earlier studies explored numerical reasoning within language comprehension contexts. For example, Dua et al. (2019) introduced a reading comprehension dataset requiring discrete reasoning, such as sorting and addition. Similarly,